

Load dynamic modules. See </usr/share/doc/nginx/README.dynamic>.

本文档主要用于记录 Doris 使用过程中的运维常见问题。会不定期更新。

文中出现的 BE 二进制文件名称 `doris_be`，在之前的版本中为 `palo_be`。

Q1. 通过 DECOMMISSION 下线 BE 节点时，为什么总会有部分 tablet 残留？

在下线过程中，通过 `show backends` 查看下线节点的 `tabletNum`，会观察到 `tabletNum` 数量在减少，说明数据分片正在从这个节点迁移走。当数量减到 0 时，系统会自动删除这个节点。但某些情况下，`tabletNum` 下降到一定数值后就不变化。这通常可能有以下两种原因：

1. 这些 tablet 属于刚被删除的表、分区或物化视图。而刚被删除的对象会保留在回收站中。而下线逻辑不会处理这些分片。可以通过修改 FE 的配置参数 `catalog_trash_expire_second` 来修改对象在回收站中驻留的时间。当对象从回收站中被删除后，这些 tablet 就会被处理了。
2. 这些 tablet 的迁移任务出现了问题。此时需要通过 `show proc "/cluster_balance"` 来查看具体任务的错误了。

对于以上情况，可以先通过 `show proc "/cluster_health/tablet_health"`；查看集群是否还有 `unhealthy` 的分片，如果为 0，则可以直接通过 `drop backend` 语句删除这个 BE。否则，还需要具体查看不健康分片的副本情况。

Q2. `priority_networks` 应该如何设置？

`priority_networks` 是 FE、BE 都有的配置参数。这个参数主要用于帮助系统选择正确的网卡 IP 作为自己的 IP。建议任何情况下，都显式的设置这个参数，以防止后续机器增加新网卡导致 IP 选择不正确的问题。

`priority_networks` 的值是 CIDR 格式表示的。分为两部分，第一部分是点分十进制的 IP 地址，第二部分是一个前缀长度。比如 `10.168.1.0/8` 会匹配所有 `10.xx.xx.xx` 的 IP 地址，而 `10.168.1.0/16` 会匹配所有 `10.168.xx.xx` 的 IP 地址。

之所以使用 CIDR 格式而不是直接指定一个具体 IP，是为了保证所有节点都可以使用统一的配置值。比如有两个节点：`10.168.10.1` 和 `10.168.10.2`，则我们可以使用 `10.168.10.0/24` 来作为 `priority_networks` 的值。

Q3. FE 的 Master、Follower、Observer 都是什么？

首先明确一点，FE 只有两种角色：Follower 和 Observer。而 Master 只是一组 Follower 节点中选择出来的一个 FE。Master 可以看成是一种特殊的 Follower。所以当我们被问及一个集群有多少 FE，都是什么角色时，正确的回答当然应该是所有 FE 节点的个数，以及 Follower 角色的个数和 Observer 角色的个数。

所有 Follower 角色的 FE 节点会组成一个可选择组，类似 Paxos 一致性协议里的组概念。组内会选举出一个 Follower 作为 Master。当 Master 挂了，会自动选择新的 Follower 作为 Master。而 Observer 不会参与选举，因此 Observer 也不会成为 Master。

一条元数据日志需要在多数 Follower 节点写入成功，才算成功。比如 3 个 FE，2 个写入成功才可以。这也是为什么 Follower 角色的个数需要是奇数的原因。

Observer 角色和这个单词的含义一样，仅仅作为观察者来同步已经成功写入的元数据日志，并且提供元数据读服务。他不会参与多数写的逻辑。

通常情况下，可以部署 1 个 Follower + 2 个 Observer 或者 3 个 Follower + N 个 Observer。前者运维简单，几乎不会出现 Follower 之间的一致性协议导致这种复杂错误情况（企业大多使用这种方式）。后者可以保证元数据写的高可用，如果是高并发查询场景，可以适当增加 Observer。

Q4. 节点新增加了新的磁盘，为什么数据没有均衡到新的磁盘上？

当前 Doris 的均衡策略是以节点为单位的。也就是说，是按照节点整体的负载指标（分片数量和总磁盘利用率）来判断集群负载。并且将数据分片从高负载节点迁移到低负载节点。如果每个节点都增加了一块磁盘，则从节点整体角度看，负载并没有改变，所以无法触发均衡逻辑。

此外，Doris 目前并不支持单个节点内部，各个磁盘间的均衡操作。所以新增磁盘后，不会将数据均衡到新的磁盘。

但是，数据在节点之间迁移时，Doris 会考虑磁盘的因素。比如一个分片从 A 节点迁移到 B 节点，会优先选择 B 节点中，磁盘空间利用率较低的磁盘。

这里我们提供 3 种方式解决这个问题：

1. 重建新表

通过 create table like 语句建立新表，然后使用 insert into select 的方式将数据从老表同步到新表。因为创建新表时，新表的数据分片会分布在新的磁盘中，从而数据也会写入新的磁盘。这种方式适用于数据量较小的情况（几十 GB 以内）。

2. 通过 Decommission 命令

decommission 命令用于安全下线一个 BE 节点。该命令会先将该节点上的数据分片迁移到其他节点，然后再删除该节点。前面说过，在数据迁移时，会优先考虑磁盘利用率低的磁盘，因此该方式可以"强制"让数据迁移到其他节点的磁盘上。当数据迁移完成后，我们再 cancel 掉这个 decommission 操作，这样，数据又会重新均衡回这个节点。当我们对所有 BE 节点都执行一遍上述步骤后，数据将会均匀的分布在所有节点的所有磁盘上。

注意，在执行 decommission 命令前，先执行以下命令，以避免节点下线完成后被删除。

```
admin set frontend config("drop_backend_after_decommission" = "false");
```

3. 使用 API 手动迁移数据

Doris 提供了 HTTP API，可以手动指定一个磁盘上的数据分片迁移到另一个磁盘上。

Q5. 如何正确阅读 FE/BE 日志？

很多情况下我们需要通过日志来排查问题。这里说明一下 FE/BE 日志的格式和查看方式。

1. FE

FE 日志主要有：

- fe.log：主日志。包括除 fe.out 外的所有内容。
- fe.warn.log：主日志的子集，仅记录 WARN 和 ERROR 级别的日志。
- fe.out：标准/错误输出的日志（stdout 和 stderr）。
- fe.audit.log：审计日志，记录这个 FE 接收的所有 SQL 请求。

一条典型的 FE 日志如下：

2021-09-16 23:13:22,502 INFO (tablet scheduler|43)
[BeLoadRebalancer.selectAlternativeTabletsForCluster():85] cluster is balance:
default_cluster with medium: HDD. skip

- 2021-09-16 23:13:22,502：日志时间。
- INFO：日志级别，默认是 INFO。
- (tablet scheduler|43)：线程名称和线程 id。通过线程 id，就可以查看这个线程上下文信息，方便排查这个线程发生的事情。
- BeLoadRebalancer.selectAlternativeTabletsForCluster():85：类名、方法名和代码行号。
- cluster is balance xxx：日志内容。

通常情况下我们主要查看 fe.log 日志。特殊情况下，有些日志可能输出到了 fe.out 中。

2. BE

BE 日志主要有：

- be.INFO：主日志。这其实是个软连，连接到最新的一个 be.INFO.xxxx 上。
- be.WARNING：主日志的子集，仅记录 WARN 和 FATAL 级别的日志。这其实是个软连，连接到最新的一个 be.WARN.xxxx 上。
- be.out：标准/错误输出的日志（stdout 和 stderr）。

一条典型的 BE 日志如下：

I0916 23:21:22.038795 28087 task_worker_pool.cpp:1594] finish report TASK. master
host: 10.10.10.10, port: 9222

- I0916 23:21:22.038795：日志等级和日期时间。大写字母 I 表示 INFO，W 表示 WARN，F 表示 FATAL。
- 28087：线程 id。通过线程 id，就可以查看这个线程上下文信息，方便排查这个线程发生的事情。
- task_worker_pool.cpp:1594：代码文件和行号。
- finish report TASK xxx：日志内容。

通常情况下我们主要查看 be.INFO 日志。特殊情况下，如 BE 宕机，则需要查看 be.out。

Q6. FE/BE 节点挂了应该如何排查原因？

1. BE

BE 进程是 C/C++ 进程，可能会因为一些程序 Bug（内存越界，非法地址访问等）或 Out Of Memory（OOM）导致进程挂掉。此时我们可以通过以下几个步骤查看错误原因：

1. 查看 be.out

BE 进程实现了在程序因异常情况退出时，会打印当前的错误堆栈到 be.out 里（注意是 be.out，不是 be.INFO 或 be.WARNING）。通过错误堆栈，通常能够大致获悉程序出错的位置。

注意，如果 be.out 中出现错误堆栈，通常情况下是因为程序 bug，普通用户可能无法自行解决，欢迎前往微信群、GitHub Discussion 或 dev 邮件组寻求帮助，并贴出对应的错误堆栈，以便快速排查问题。

2. dmesg

如果 be.out 没有堆栈信息，则大概率是因为 OOM 被系统强制 kill 掉了。此时可以通过 dmesg -T 这个命令查看 Linux 系统日志，如果最后出现 Memory cgroup out of memory: Kill process 7187 (doris_be) score 1007 or sacrifice child 类似的日志，则说明是 OOM 导致的。

内存问题可能有多方面原因，如大查询、导入、compaction 等。Doris 也在不断优化内存使用。欢迎前往微信群、GitHub Discussion 或 dev 邮件组寻求帮助。

3. 查看 be.INFO 中是否有 F 开头的日志

F 开头的日志是 Fatal 日志。如 F0916，表示 9 月 16 号的 Fatal 日志。Fatal 日志通常表示程序断言错误，断言错误会直接导致进程退出（说明程序出现了 Bug）。欢迎前往微信群、GitHub Discussion 或 dev 邮件组寻求帮助。

2. FE

FE 是 Java 进程，健壮程度要优于 C/C++ 程序。通常 FE 挂掉的原因可能是 OOM (Out-of-Memory) 或者是元数据写入失败。这些错误通常在 fe.log 或者 fe.out 中有错误堆栈。需要根据错误堆栈信息进一步排查。

Q7. 关于数据目录 SSD 和 HDD 的配置，建表有时候会遇到报错 Failed to find enough host with storage medium and tag

Doris 支持一个 BE 节点配置多个存储路径。通常情况下，每块盘配置一个存储路径即可。同时，Doris 支持指定路径的存储介质属性，如 SSD 或 HDD。SSD 代表高速存储设备，HDD 代表低速存储设备。

如果集群只有一种介质比如都是 HDD 或者都是 SSD，最佳实践是不用在 be.conf 中显式指定介质属性。如果遇到上述报错 Failed to find enough host with storage medium and tag，一般是因为 be.conf 中只配置了 SSD 的介质，而建表阶段中显式指定了 properties {"storage_medium" = "hdd"}；同理如果 be.conf 只配置了 HDD 的介质，而建表阶段中显式指定了 properties {"storage_medium" = "ssd"} 也会出现上述错误。解决方案可以修改建表的 properties 参数与配置匹配；或者将 be.conf 中 SSD/HDD 的显式配置去掉即可。

通过指定路径的存储介质属性，我们可以利用 Doris 的冷热数据分区存储功能，在分区级别将热数据存储 SSD 中，而冷数据会自动转移到 HDD 中。

需要注意的是，Doris 并不会自动感知存储路径所在磁盘的实际存储介质类型。这个类型需要用户在路径配置中显式的表示。比如路径 /path/to/data1.SSD 即表示这个路径是 SSD 存储介质。而 data1.SSD 就是实际的目录名称。Doris 是根据目录名称后面的 .SSD 后缀来确定存储介质类型的，而不是实际的存储介质类型。也就是说，用户可以指定任意路径为 SSD 存储介质，而 Doris 仅识别目录后缀，不会去判断存储介质是否匹配。如果不写后缀，则默认为 HDD。

换句话说，.HDD 和 .SSD 只是用于标识存储目录"相对"的"低速"和"高速"之分，而并不是标识实际的存储介质类型。所以如果 BE 节点上的存储路径没有介质区别，则无需填写后缀。

Q8. 多个 FE，在使用 Nginx 实现 Web UI 负载均衡时，无法登录

Doris 可以部署多个 FE，在访问 Web UI 的时候，如果使用 Nginx 进行负载均衡，因为 Session 问题会出现不停的提示要重新登录，这个问题其实是 Session 共享的问题，Nginx 提供了集中式 Session 共享的解决方案，这里我们使用的是 Nginx 中的 ip_hash 技术，ip_hash 能够将某个 IP 的请求定向到同一台后端，这样一来这个 IP 下的某个客户端和某个后端就能建立起稳固的 Session，ip_hash 是在 upstream 配置中定义的：

```
upstream doris.com {  
    server 172.22.197.238:8030 weight=3;  
    server 172.22.197.239:8030 weight=4;  
    server 172.22.197.240:8030 weight=4;  
    ip_hash;  
}
```

完整的 Nginx 示例配置如下：

```
user nginx;  
worker_processes auto;  
error_log /var/log/nginx/error.log;  
pid /run/nginx.pid;
```

```
# Load dynamic modules. See /usr/share/doc/nginx/README.dynamic.  
include /usr/share/nginx/modules/*.conf;
```

```
events {  
    worker_connections 1024;  
}
```

```
http {  
    log_format main '$remote_addr - $remote_user [$time_local] "$request" '  
        '$status $body_bytes_sent "$http_referer" '  
        '"$http_user_agent" "$http_x_forwarded_for"';  
}
```

```
access_log /var/log/nginx/access.log main;
```

```
sendfile        on;  
tcp_nopush      on;  
tcp_nodelay      on;  
keepalive_timeout 65;  
types_hash_max_size 2048;
```

```
include /etc/nginx/mime.types;  
default_type application/octet-stream;
```

```
# Load modular configuration files from the /etc/nginx/conf.d directory.  
# See http://nginx.org/en/docs/nginx\_core\_module.html#include  
# for more information.
```

```

include /etc/nginx/conf.d/*.conf;
#include /etc/nginx/custom/*.conf;
upstream doris.com {
    server 172.22.197.238:8030 weight=3;
    server 172.22.197.239:8030 weight=4;
    server 172.22.197.240:8030 weight=4;
    ip_hash;
}

server {
    listen 80;
    server_name gaia-pro-bigdata-fe02;
    if ($request_uri ~ _load) {
        return 307 http://$host$request_uri;
    }

    location / {
        proxy_pass http://doris.com;
        proxy_redirect default;
    }
    error_page 500 502 503 504 /50x.html;
    location = /50x.html {
        root html;
    }
}

```

Q9. FE 启动失败，fe.log 中一直滚动 "wait catalog to be ready. FE type UNKNOWN"

这种问题通常有两个原因：

1. 本次 FE 启动时获取到的本机 IP 和上次启动不一致，通常是因为没有正确设置 priority_network 而导致 FE 启动时匹配到了错误的 IP 地址。需修改 priority_network 后重启 FE。
2. 集群内多数 Follower FE 节点未启动。比如有 3 个 Follower，只启动了一个。此时需要将另外至少一个 FE 也启动，FE 可选举组方能选举出 Master 以提供服务。

如果以上情况都不能解决，可以按照 Doris 官网文档中的元数据运维文档进行恢复。

Q10. Lost connection to MySQL server at 'reading initial communication packet', system error: 0

如果使用 MySQL 客户端连接 Doris 时出现如下问题，这通常是因为编译 FE 时使用的 JDK 版本和运行 FE 时使用的 JDK 版本不同导致的。注意使用 Docker 编译镜像编译时，默认的 JDK 版本是 OpenJDK 11，可以通过命令切换到 OpenJDK 8（详见编译文档）。

Q11. recoveryTracker should overlap or follow on disk last VLSN of 4,422,880 recoveryFirst= 4,422,882 UNEXPECTED_STATE_FATAL

有时重启 FE，会出现如上错误（通常只会出现在多 Follower 的情况下）。并且错误中的两个数值相差 2，导致 FE 启动失败。

这是 BDB JE 的一个 bug，尚未解决。遇到这种情况，只能通过元数据运维文档中的故障恢复进行操作来恢复元数据了。

Q12. Doris 编译安装 JDK 版本不兼容问题

在自己使用 Docker 编译 Doris 的时候，编译完成安装以后启动 FE，出现 `java.lang.NoSuchMethodError: java.nio.ByteBuffer.limit(I)Ljava/nio/ByteBuffer;` 异常信息，这是因为 Docker 里默认是 JDK 11，如果你的安装环境是使用 JDK 8，需要在 Docker 里 JDK 环境切换成 JDK 8，具体切换方法参照编译文档。

Q13. 本地启动 FE 或者启动单元测试报错 Cannot find external parser table action_table.dat

执行如下命令：

```
cd fe && mvn clean install -DskipTests
```

如果还报同样的错误，手动执行如下命令：

```
cp fe-core/target/generated-sources/cup/org/apache/doris/analysis/action_table.dat fe-core/target/classes/org/apache/doris/analysis
```

Q14. 升级到 1.2 版本，BE NoClassDefFoundError 问题启动失败

:::note Java UDF 依赖错误从 Doris 1.2 版本后开始支持 :::

如果升级后启动 BE 出现下面这种 Java NoClassDefFoundError 错误：

```
Exception in thread "main" java.lang.NoClassDefFoundError: org/apache/doris/udf/JniUtil
Caused by: java.lang.ClassNotFoundException: org.apache.doris.udf.JniUtil
```

需要从官网下载 apache-doris-java-udf-jar-with-dependencies-1.2.0 的 Java UDF 函数依赖包，放到 BE 安装目录下的 lib 目录，然后重新启动 BE。

Q15. 升级到 1.2 版本，BE 启动显示 Failed to initialize JNI 问题

:::note Java 环境问题从 Doris 1.2 版本后开始支持 :::

如果升级后启动 BE 出现下面这种 Failed to initialize JNI 错误：

Failed to initialize JNI: Failed to find the library libjvm.so.

需要在系统设置 JAVA_HOME 环境变量，或者在 be.conf 中设置 JAVA_HOME 变量，然后重新启动 BE 节点。